

《面向对象程序设计实践》

课程论文

题目：	通讯录管理系统
专业：	软件工程
班级：	2024 级 5 班
小组成员 1：	陈智杰 202425220506
2：	肖睿翔 202425220524
指导老师：	彭红星
提交时间：	2026 年 5 月 17 日

目录

- 1 系统分析 3
 - 1.1 问题描述 3
 - 1.2 系统功能分析 3
 - 1.3 开发平台及工具介绍 3
- 2 系统设计 4
 - 2.1 系统总体结构设计 4
 - 2.2 系统各个类及类之间关系设计 4
 - 2.3 数据存储的设计 6
 - 2.4 界面设计..... 6
- 3 系统实现 6
 - 3.1 核心模块流程图 7
 - 3.2 关键代码片段展示 16
- 4 系统测试 20
- 5 系统运行界面 24
- 6 总结 27
 - 6.1 个人承担工作 27
 - 6.2 课程设计体会 28

1 系统分析

1.1 问题描述

随着日常人际交往增多，个人联系人信息数量不断增加，传统纸质通讯录易丢失、不易查找、不便批量管理。为解决联系人信息新增、查询、修改、删除、分类、保存等痛点，基于面向对象思想开发一款桌面版通讯录管理系统。

系统需实现对联系人姓名、电话、性别、邮箱、备注等信息的统一管理，支持数据持久化保存，操作简洁、逻辑清晰，具备良好的易用性和可扩展性，满足个人日常通讯录信息化管理需求。

1.2 系统功能分析

本通讯录管理系统采用模块化设计，核心功能如下：

(1) 联系人管理，包括：

- 增加联系人，输入并存储新的联系人；
- 修改联系人，修改已经存在的联系人的资料；
- 删除联系人：删除已经存在的联系人的资料。

(2) 联系组管理，包括：

- 增加分组，增加一个新的分组；
- 删除分组，删除一个已经存在的分组。注意：删除分组时不能把该组的联系人删除而是设置联系人不在该组。

(3) 联系人分组调整，包括：

- 把一个或多个联系人放入一个或多个分组中；
- 把一个或多个联系人从某个分组中移除。

(4) 查询联系人，在界面的联系人分组中选择全部联系人或某个分组后，列出全部或该组中的联系人。程序应该支持调整可以显示在列表中的联系人的属性。

(5) 搜索联系人，界面中的搜索框中输入（姓名、电话号码、手机、姓名的汉语拼音的声母或全拼）后列出符合条件的联系人列表。

支持模糊查询即输入部分数据后，可以查询出符合条件的全部条目。如：输入“张”，列出所有名字中有“张”的条目；输入“2645”列出电话中有该 4 个数字的所有条目；输入“xl”，列出声母为该 2 个字母的所有条目

说明：查询结果如果有多个条目，应按照姓名排序，并使用姓或姓的第一个汉语拼音的字母分类。

(6) 通讯录内容的导入和导出功能，包括：

- 通讯录的全部或部分内容导出为 CSV 格式和 vCard 格式的文件；
- 导入 CSV 格式和 vCard 格式中的联系人数据，并追加到程序的通讯录中。程序应该能够与主流手机或其他通讯录软件的 vCard 文件兼容。

1.3 开发平台及工具介绍

1.3.1 开发语言

采用 Java 语言开发，遵循面向对象三大特性：封装、继承、多态，适合本次面向对象程序设计实践要求。

1.3.2 开发环境

操作系统: Windows 11

开发 IDE: IntelliJ IDEA 2023

JDK 版本: JDK 21

构建工具: Apache Maven 3.9

1.3.3 数据存储工具

采用本地 XML 文件存储联系人数据,存放于用户主目录下的 ContactManager 文件夹。XML 格式结构清晰、可读性强、支持嵌套分组信息,满足课程设计数据持久化需求。同时支持导入导出 CSV 和 vCard 格式以实现与外部系统的数据交换。

1.3.4 界面实现

使用 Java JavaFX 框架搭建图形化可视化界面,实现按钮、输入框、表格、下拉框、对话框等交互组件,操作直观易懂。JavaFX 相比 Swing 提供了更现代的 UI 组件和 CSS 样式支持,界面美观度更高。

2 系统设计

2.1 系统总体结构设计

本系统采用分层模块化架构,整体分为界面层、业务逻辑层、实体类层、数据持久化层四层结构:

1. 实体类层:封装联系人基础属性,定义联系人实体结构。
2. 业务逻辑层:封装增删改查、数据校验、业务规则处理核心逻辑。
3. 界面层:基于 JavaFX 设计主界面、联系人编辑对话框等交互页面。
4. 数据持久化层:负责读取本地 Txt 文件、写入数据、加载历史联系人信息。

系统整体流程:用户界面操作→调用业务逻辑方法→实体类封装数据→持久化层读写文件→反馈结果到界面。

项目包结构如下:

```
com.contactmanager.App —— 程序入口 (继承 JavaFX Application)

com.contactmanager.model.Contact —— 联系人实体类

com.contactmanager.model.ContactBook —— 通讯录数据模型 (含搜索逻辑)

com.contactmanager.service.StorageService —— 数据存储服务

com.contactmanager.ui.MainWindow —— 主界面

com.contactmanager.ui.ContactEditDialog —— 联系人编辑对话框

com.contactmanager.util.PinyinUtil —— 拼音工具类
```

2.2 系统各个类及类之间关系设计

2.2.1 核心实体类

1. Contact（联系人类）

封装属性：姓名、手机号、性别、邮箱、地址、备注、电话、即时通信工具、个人主页、生日、像片、工作单位、邮编；提供构造方法、get/set 方法、重写 toString 方法，实现数据封装。

字段名	类型	说明
id	String	联系人唯一标识（UUID）
name	String	姓名（必填）
phone	String	固定电话
mobile	String	手机号码
imTool	String	即时通信工具（如 QQ、微信等）
imNumber	String	即时通信账号
email	String	电子邮箱
homepage	String	个人主页 URL
birthday	String	生日（yyyy-MM-dd）
photoPath	String	头像图片本地路径
company	String	工作单位
address	String	家庭地址
zipCode	String	邮政编码
groups	Set<String>	所属分组集合（可多组）
remark	String	备注
pinyin	String	姓名拼音全拼（用于搜索）
pinyinInitials	String	姓名拼音声母（用于搜索）

主要方法：addGroup()、removeGroup()、isInGroup()、getFirstLetter()、compareTo()。

2. ContactManager（通讯录管理工具类）

包含联系人集合 List，提供添加、删除、修改、查询、获取所有联系人等核心业务方法，是系统业务调度核心。

2.2.2 文件操作类

FileUtil（文件工具类）

封装文件读取、文件写入、文件初始化、数据解析保存方法，负责所有本地数据持久化操作。

2.2.3 界面窗体类

MainFrame、AddFrame、EditFrame、SearchFrame

继承 JFrame 窗体类，依赖管理类 and 文件工具类，完成界面组件初始化、事件监听、数据交

互。

2.2.4 类之间关系

- Contact 实体类被管理类、界面类依赖调用；
- ContactManager 依赖 FileUtil 实现数据读写；
- 所有界面窗体类依赖 ContactManager 完成业务功能调用；
- 整体符合面向对象封装、低耦合、高内聚设计思想。

2.3 数据存储的设计

1. 存储方式：采用纯文本 Txt 文件存储，文件命名为 contact.txt，存放于项目根目录。
2. 存储格式：每条联系人信息单独一行，字段用逗号分隔，格式为：姓名,手机号,性别,邮箱,地址,备注。
3. 读写规则：程序启动时自动读取 XML 文件加载所有联系人；增删改操作后调用 save() 同步写入文件，保证数据实时持久化。
4. 异常处理：若文件不存在，程序自动创建空白文件，避免程序崩溃。
5. CSV 格式：每行一条联系人，字段用逗号分隔，首行为标题，支持引号转义。
6. vCard 格式：遵循 RFC 6350 (vCard 3.0) 标准，字段包括 FN、TEL、EMAIL、ORG、ADR、BDAY、URL、NOTE、CATEGORIES，与 QQ、iOS 通讯录兼容。

2.4 界面设计

主界面采用 BorderPane 布局，分为以下区域：

顶部：深蓝色工具栏，包含程序标题、搜索框（实时模糊搜索）、菜单栏（联系人/分组/导入导出）。

左侧：深色分组面板（宽 180px），显示“全部联系人”“未分组”及自定义分组，底部有“新增分组”按钮。

中间：白色联系人列表区，顶部有新增/编辑/删除操作按钮，列表以 TableView 展示，列包括头像、姓名、手机、电话、邮箱、单位、分组。双击行可编辑。

底部：灰色状态栏，显示当前显示联系人数、总数及数据文件路径。

联系人编辑对话框采用 TabPane 三标签页设计：

基本信息：左侧圆形头像，右侧依次填写姓名、手机、固话、即时通信工具+账号、邮箱、生日、单位。

详细信息：个人主页、家庭地址、邮编、多行备注。

所属分组：以复选框列出所有分组，可多选。

3 系统实现

3.1 核心模块流程图

3.1.1 系统启动流程

启动程序→初始化窗体→调用文件工具类读取 xml 文件→加载联系人到集合→主界面表格渲染数据→等待用户操作。

```
// 加载通讯录数据
ContactBook loadedBook;
try {
    loadedBook = storageService.load();
} catch (Exception e) {
    System.err.println("加载数据失败，使用空通讯录：" + e.getMessage());
    loadedBook = new ContactBook();
    addSampleData(loadedBook);
}
// 用 final 变量，使 lambda 可以引用
```

一段用于将保存的数据载入

顺序图

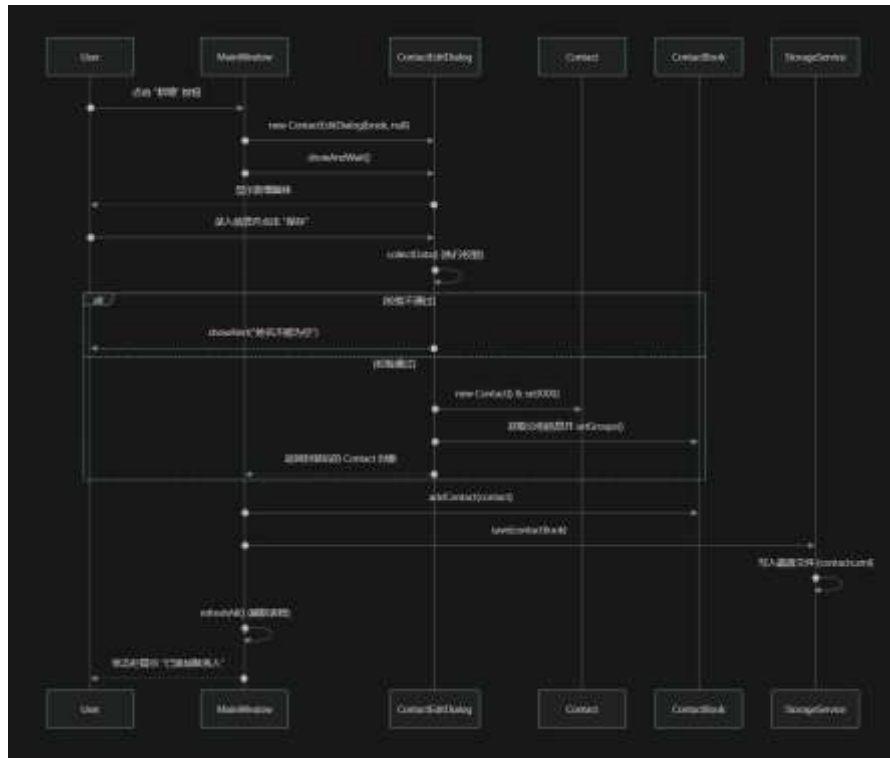


3.1.2 新增联系人流程

点击新增按钮→弹出新增窗体→录入信息→格式校验→校验通过封装为 Contact 对象→加入集合→写入 xml 文件→刷新主界面表格。

```
public void removeContact(Contact c) {
```

顺序图



选中表格联系人→点击删除按钮→弹窗确认→移除集合中对应对象→更新本地文件→刷新界面。

- 与新增联系人原理一致，就是从可变数组把他删掉

顺序图



3.1.4 修改联系人

选择与进入->初始化对话框->信息修改->提交修改->更新内存与文件->界面刷新

```

public void updateContact(Contact updated) {
    for (int i = 0; i < contacts.size(); i++) {
        if (contacts.get(i).getId().equals(updated.getId())) {
            contacts.set(i, updated);
            return;
        }
    }
}
  
```

-用于将收集到的信息更新

顺序图



3.1.5 增加联系人分组

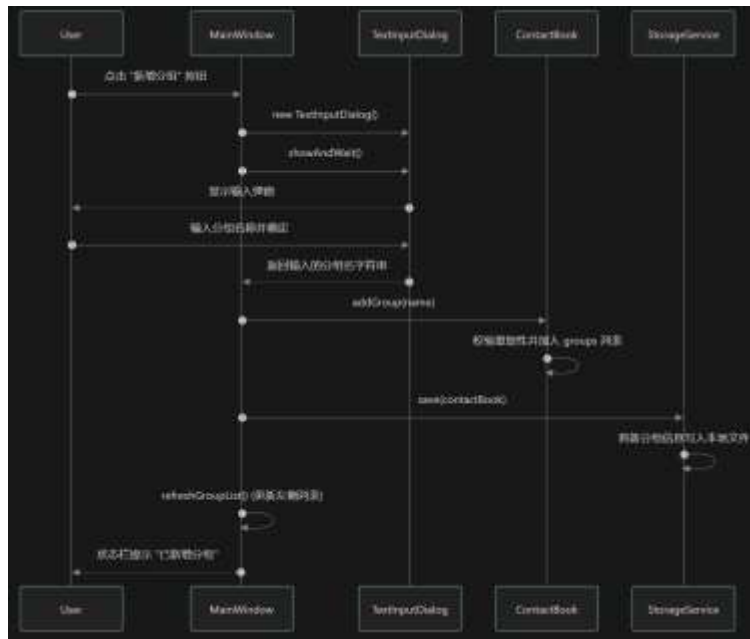
触发操作->输入分组名->内存更新->文件同步->界面刷新

```

public void addGroup(String groupName) {
    if (!groups.contains(groupName)) {
        groups.add(groupName);
    }
}

```

与联系人逻辑一样，把 group 放进一个可变数组里
顺序图



3.1.6 删除联系人分组

触发与验证->二次确认->业务逻辑执行->持久化->界面刷新

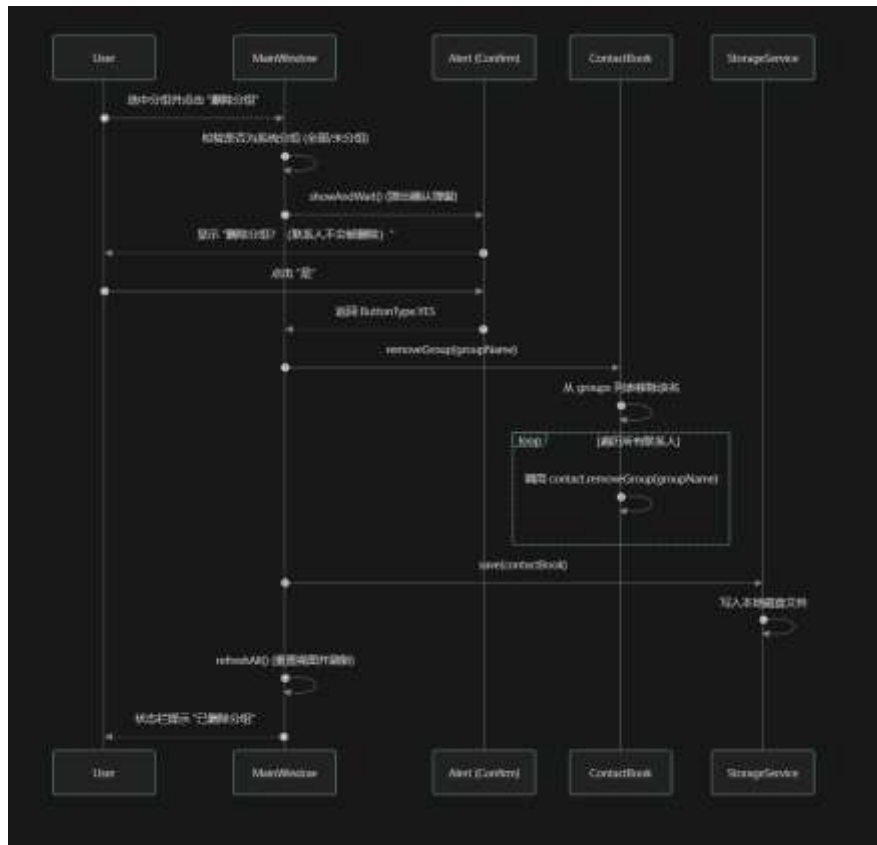
```

/*
 * 删除分组，但不删除联系人，只是将联系人移出该组
 */
public void removeGroup(String groupName) {
    groups.remove(groupName);
    for (Contact c : contacts) {
        c.removeGroup(groupName);
    }
}

```

-与联系人逻辑一样，把可变数组里的指定 group 去掉

顺序图



3. 1. 7 把一个或多个联系人放入一个或多个分组

选择联系人->触发操作->选择目标分组->执行分配->持久化与刷新

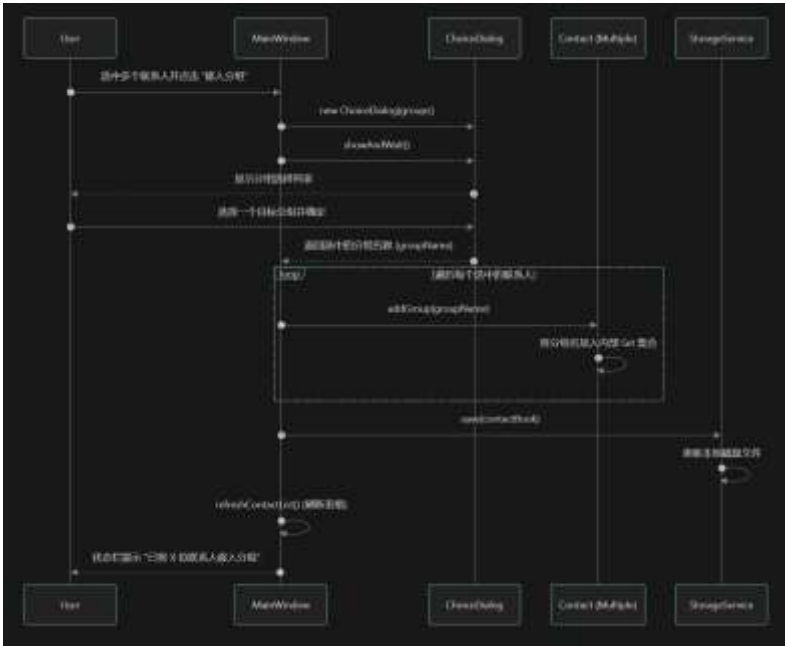
```

private void moveToGroup() {
    List<Contact> selected = new ArrayList<>(contactTable.getSelectionModel());
    if (selected.isEmpty()) { showInfo(msg: "请先选择联系人"); return; }

    List<String> groups = contactBook.getGroups();
    if (groups.isEmpty()) { showInfo(msg: "还没有分组，请先新增分组"); return; }

    ChoiceDialog<String> dlg = new ChoiceDialog<>(groups.get(0), groups);
    dlg.setTitle("移入分组");
    dlg.setHeaderText("选择目标分组: ");
    dlg.setContentText("分组: ");
    dlg.initOwner(stage);
    Optional<String> res = dlg.showAndWait();
    res.ifPresent(group -> {
        for (Contact c : selected) c.addGroup(group);
        saveData();
        refreshContactList();
        showStatus("已将 " + selected.size() + " 位联系人移入分组: " + group);
    });
}
  
```

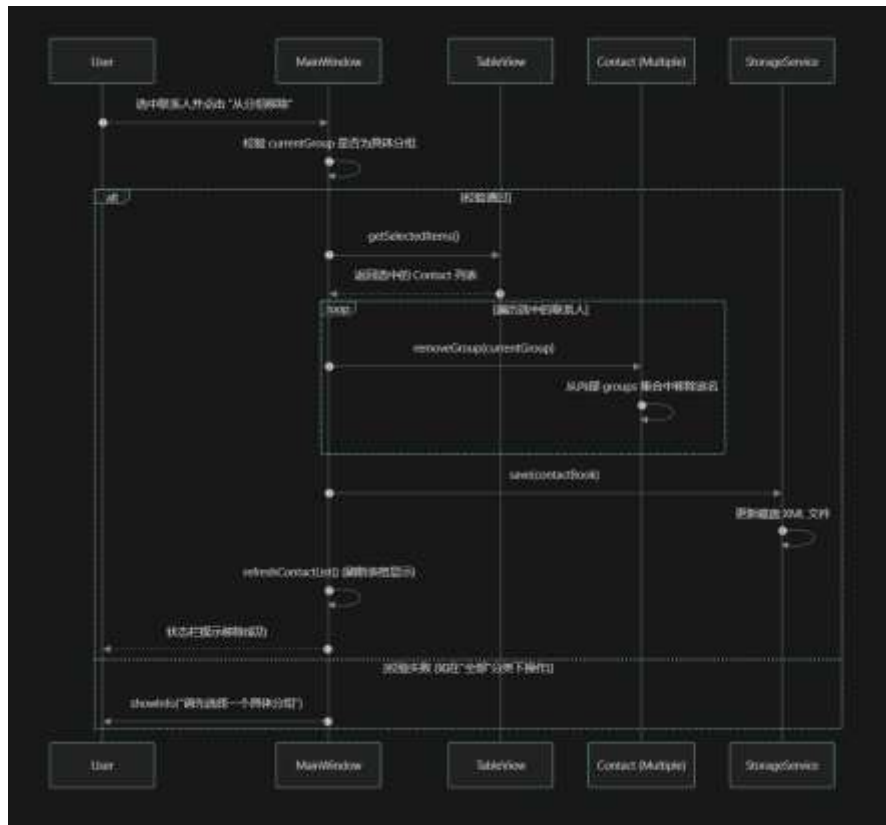
-实现在组中批量加入组员的片段，界面层
顺序图



3. 1. 8 把一个或多个联系人从某个分组中移除
环境准备-触发操作-合法性校验-执行移除-持久化-界面刷新

```
private void removeFromGroup() {
    List<Contact> selected = new ArrayList<>(contactTable.getSelectionModel);
    if (selected.isEmpty()) { showInfo(msg: "请先选择联系人"); return; }
    if (currentGroup.equals("全部联系人") || currentGroup.equals("未分组")) {
        showInfo(msg: "请先在左侧选择一个具体分组");
        return;
    }
    for (Contact c : selected) c.removeGroup(currentGroup);
    saveData();
    refreshContactList();
    showStatus("已从分组「" + currentGroup + "」移除 " + selected.size() + "
}
```

-实现在组中批量删除组员的片段，界面层
顺序图



3.1.9 查询联系人

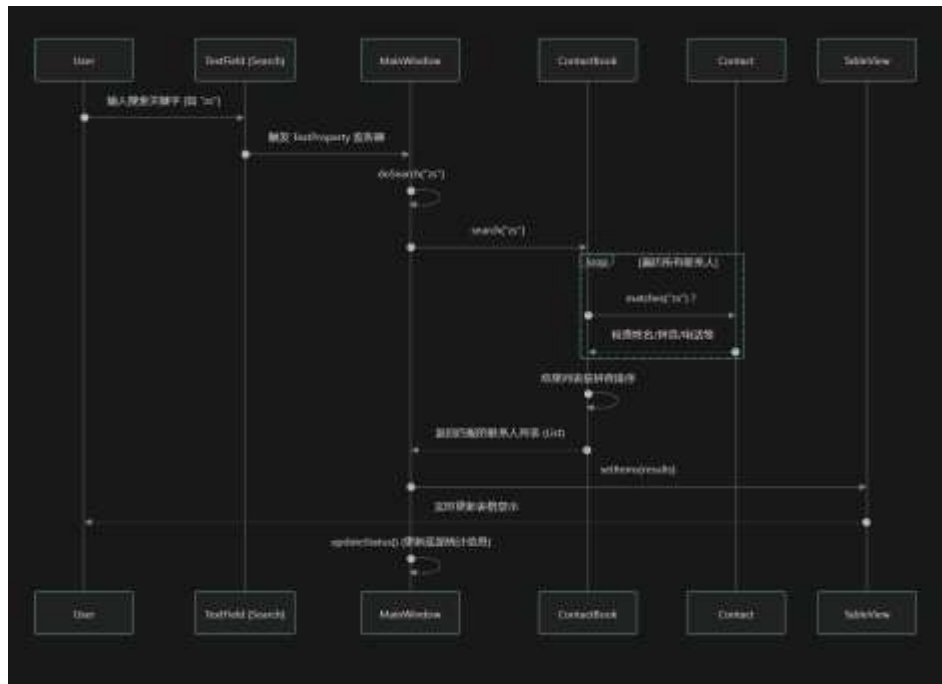
触发搜索-执行匹配逻辑-结果排序-界面渲染-状态更新

```

/**
 * 搜索联系人：支持姓名、电话、手机、拼音字母、字母缩写搜索
 */
public List<Contact> search(String keyword) {
    if (keyword == null || keyword.trim().isEmpty()) return getContactsByGroup(groupName: "全部联系人");
    String kw = keyword.trim().toLowerCase();
    List<Contact> result = new ArrayList<>();
    for (Contact c : contacts) {
        if (matches(c, kw)) result.add(c);
    }
    Collections.sort(result);
    return result;
}

private boolean matches(Contact c, String kw) {
    // 姓名
    if (c.getName() != null && c.getName().toLowerCase().contains(kw)) return true;
    // 手机
    if (c.getMobile() != null && c.getMobile().contains(kw)) return true;
    // 固定电话
    if (c.getPhone() != null && c.getPhone().contains(kw)) return true;
    // 拼音字母
    if (c.getPinyin() != null && c.getPinyin().toLowerCase().contains(kw)) return true;
    // 首字母
    if (c.getPinyinInitials() != null && c.getPinyinInitials().toLowerCase().contains(kw)) return true;
    // 邮箱
    if (c.getEmail() != null && c.getEmail().toLowerCase().contains(kw)) return true;
    // 单位
    if (c.getCompany() != null && c.getCompany().toLowerCase().contains(kw)) return true;
    return false;
}
  
```

-这一大段都是查询逻辑，其中用 util.PinyinUtil 实现了拼音转汉字搜索
顺序图

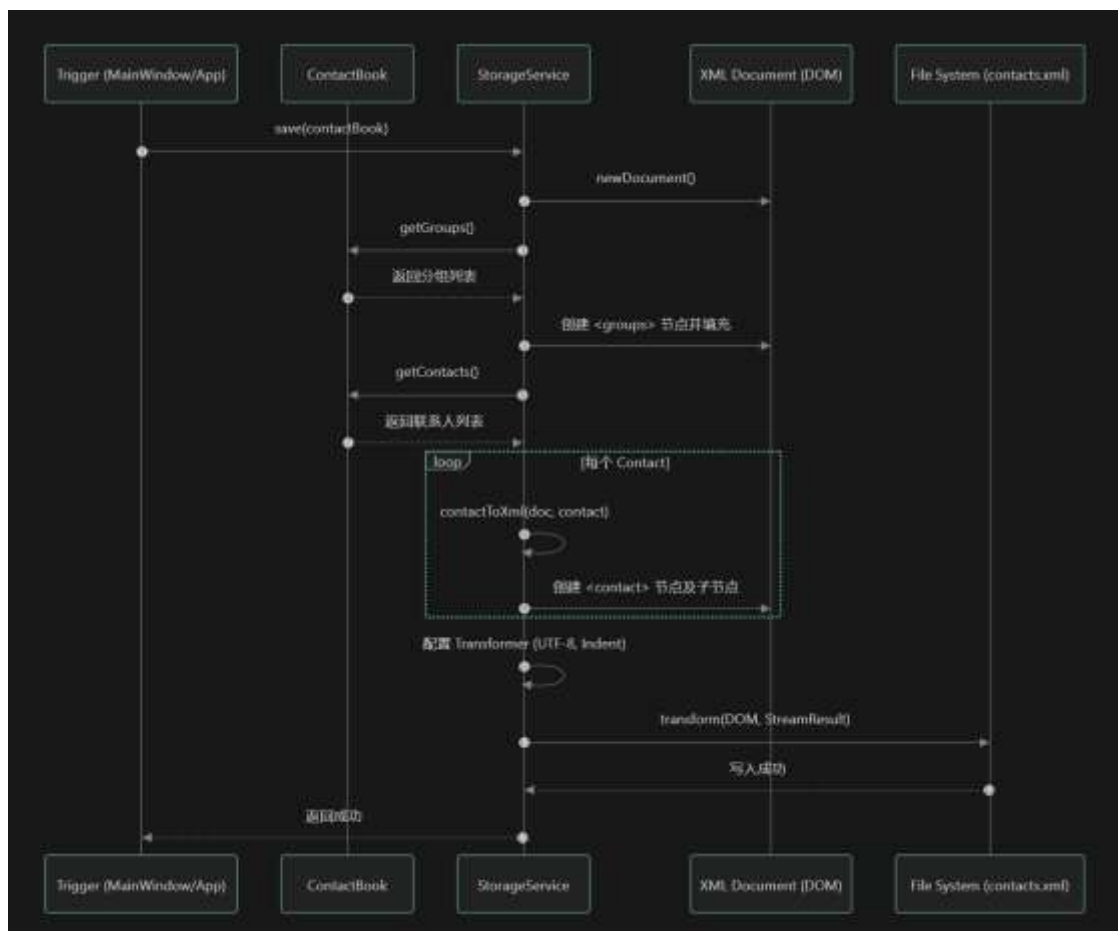


3. 1. 10 保存逻辑

触发保存-准备数据-构建 xml 文档-序列化

-保存逻辑写在 StorageService.java 内

顺序图



3.2 关键代码片段展示

3.2.1 Contact 实体类（部分）

Contact 类实现 Comparable 接口，按拼音字母排序，实现如下：

```
public class Contact implements Serializable, Comparable<Contact> {  
    private String id;        // 唯一标识 UUID  
    private String name;      // 姓名  
    private String mobile;    // 手机  
    private String phone;     // 固话  
    private String email;     // 邮箱  
    private Set<String> groups; // 所属分组（可多组）  
    private String pinyin;     // 拼音全拼（搜索用）  
    private String pinyinInitials; // 声母（搜索用）  
    // ... 其余字段省略  
  
    @Override  
    public int compareTo(Contact other) {  
        String a = this.pinyin != null ? this.pinyin : this.name;  
        String b = other.pinyin != null ? other.pinyin : other.name;  
        return a.compareToIgnoreCase(b);  
    }  
}
```

3.2.2 模糊搜索实现（ContactBook.search）

搜索方法同时匹配多个字段，支持拼音声母搜索：

```
public List<Contact> search(String keyword) {  
    String kw = keyword.trim().toLowerCase();  
    List<Contact> result = new ArrayList<>();  
    for (Contact c : contacts) {  
        if (matches(c, kw)) result.add(c);  
    }  
    Collections.sort(result); // 按拼音排序  
    return result;  
}
```



```

    }

    private boolean matches(Contact c, String kw) {
        if(c.getName() != null && c.getName().toLowerCase().contains(kw))
            return true;
        if(c.getMobile() != null && c.getMobile().contains(kw))
            return true;
        if(c.getPhone() != null && c.getPhone().contains(kw))
            return true;

        if(c.getPinyin() != null && c.getPinyin().toLowerCase().contains(kw))
            return true;
        if (c.getPinyinInitials() != null
            && c.getPinyinInitials().toLowerCase().contains(kw))
            return true;

        if(c.getEmail() != null && c.getEmail().toLowerCase().contains(kw))
            return true;

        if(c.getCompany() != null && c.getCompany().toLowerCase().contains(kw))
            return true;
        return false;
    }
}

```

3.2.3 XML 数据存储 (StorageService.save)

使用 Java 内置 javax.xml 包实现 XML 序列化, 无需第三方依赖:

```

public void save(ContactBook book) throws Exception {
    DocumentBuilder db =
        DocumentBuilderFactory.newInstance().newDocumentBuilder();
    Document doc = db.newDocument();
    Element root = doc.createElement("contactbook");
    doc.appendChild(root);
    // 保存分组
    Element groupsEl = doc.createElement("groups");
    for (String g : book.getGroups()) {

```

```

        Element ge = doc.createElement("group");
        ge.setTextContent(g);
        groupsEl.appendChild(ge);
    }
    root.appendChild(groupsEl);
    // 保存联系人
    Element contactsEl = doc.createElement("contacts");
    for (Contact c : book.getContacts()) {
        contactsEl.appendChild(contactToXml(doc, c));
    }
    root.appendChild(contactsEl);
    // 写入文件 (UTF-8, 缩进格式)
    Transformer tf =
TransformerFactory.newInstance().newTransformer();
    tf.setOutputProperty(OutputKeys.INDENT, "yes");
    tf.setOutputProperty(OutputKeys.ENCODING, "UTF-8");
    tf.transform(new DOMSource(doc), new StreamResult(new
File(DATA_FILE)));
}

```

3.2.4 删除分组 (ContactBook.removeGroup)

删除分组时不删除联系人，只将联系人移出该组：

```

public void removeGroup(String groupName) {
    groups.remove(groupName); // 从分组列表移除
    for (Contact c : contacts) {
        c.removeGroup(groupName); // 联系人移出该组，但联系人本身不删除
    }
}

```

3.2.5 vCard 导出 (StorageService.exportVCard)

遵循 vCard 3.0 标准，生成手机可直接导入的 .vcf 文件：

```

public void exportVCard(List<Contact> contacts, String filePath)

```

```

throws IOException {
    try (PrintWriter pw = new PrintWriter(
        new OutputStreamWriter(new FileOutputStream(filePath),
            "UTF-8"))) {
        for (Contact c : contacts) {
            pw.println("BEGIN:VCARD");
            pw.println("VERSION:3.0");
            pw.println("FN:" + safe(c.getName()));
            if (c.getMobile() != null) pw.println("TEL;TYPE=CELL:" +
c.getMobile());
            if (c.getPhone() != null) pw.println("TEL;TYPE=HOME:" +
c.getPhone());
            if (c.getEmail() != null) pw.println("EMAIL:" +
c.getEmail());
            if (c.getCompany() != null) pw.println("ORG:" +
c.getCompany());
            if (c.getBirthday() != null) pw.println("BDAY:" +
c.getBirthday());
            if (c.getGroups() != null)
                for (String g : c.getGroups())
                    pw.println("CATEGORIES:" + g);
            pw.println("END:VCARD");
            pw.println();
        }
    }
}

```

3.2.6 JavaFX 事件处理示例 (MainWindow)

JavaFX 使用 Lambda 表达式简化事件处理，属性监听器实现实时搜索：

```

// 按钮点击事件
addBtn.setOnAction(e -> addContact());
deleteBtn.setOnAction(e -> deleteContact());

// 搜索框实时监听——每次按键都触发搜索
searchField.textProperty().addListener((obs, oldVal, newVal) -> {

```

```

        doSearch(newVal);
    });

// 表格双击编辑
row.setOnMouseClicked(e -> {
    if (e.getButton() == MouseButton.PRIMARY
        && e.getClickCount() == 2 && !row.isEmpty()) {
        editContact();
    }
});

// ObservableList 与 TableView 绑定—数据变化 UI 自动刷新
ObservableList<Contact> contactItems =
FXCollections.observableArrayList();
contactTable.setItems(contactItems);
contactItems.setAll(newContactList); // 刷新只需这一行

```

4 系统测试

4.1 新增联系人测试

测试项	输入数据	预期结果	实际结果 / 结论
新增—正常	姓名：张三；手机：13800138000；邮箱：zhangsan@qq.com；分组：朋友	联系人成功保存到列表和 XML 文件，状态栏计数+1	通过：联系人显示在列表，contacts.xml 新增对应记录
新增—姓名为空	不填姓名，填手机13900000001，点保存	弹出警告“姓名不能为空”，不保存	通过：弹出警告对话框，记录未写入

4.2 修改联系人测试

测试项	输入数据	预期结果	实际结果 / 结论
修改手机	双击“张三”，将手机改为18800000001，保存	列表和 XML 中张三手机号更新	通过：表格即时刷新，XML 同步更新
修改分组	编辑“张三”，在分组页取消勾选“朋友”，勾选“同事”，保存	张三从“朋友”移入“同事”分组	通过：分组列表切换后显示正确

4.3 删除联系人测试

测试项	输入数据	预期结果	实际结果 / 结论
删除单个	选中“张三”，点删除，确认弹窗	张三从列表和 XML 文件移除，计数-1	通过：记录消失，文件同步
删除多个	Ctrl+点击选中多条，点删除，确认	所选联系人全部删除	通过：批量删除正常

4.4 分组管理测试

测试项	输入数据	预期结果	实际结果 / 结论
新增分组	点菜单“分组”→新增分组“”，输入“大学同学”	左侧分组列表新增“大学同学”	通过：分组立即显示
删除分组	选中“大学同学”，点菜单“分组”→删除分组“”，确认	分组删除，但原组内联系人仍存在于“全部联系人”	通过：联系人未被删除，移至“未分组”
联系	选中多个联系人，右键→“移入分组”→选“朋友”	所选联系人均加入“朋友”分组	通过：切换至“朋友”分组可见

测试项	输入数据	预期结果	实际结果 / 结论
人加入分组	”		
联系人移出分组	在”朋友”分组下选中联系人，菜单→”从分组移除”	联系人从”朋友”分组移除，仍在”全部联系人”	通过：分组计数减少，联系人未被删除

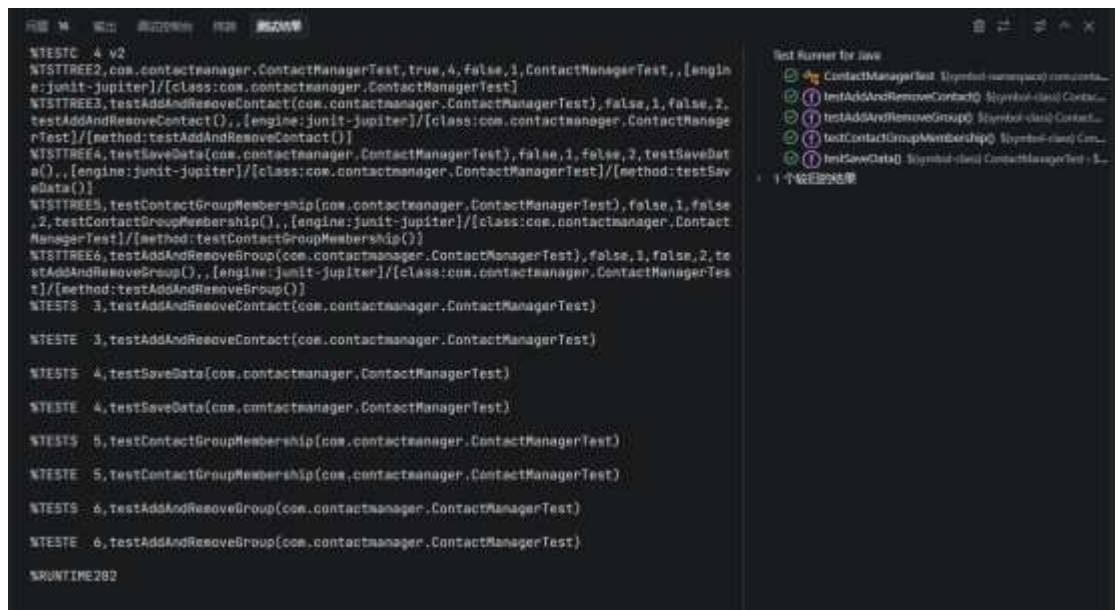
4.5 搜索测试

测试项	输入数据	预期结果	实际结果 / 结论
汉字模糊搜索	搜索框输入”张”	列出所有姓名含”张”的联系人，按拼音排序	通过：结果正确，排序正确
手机号模糊搜索	输入”138”	列出手机号含”138”的所有联系人	通过
拼音全拼搜索	输入”zhang”	列出姓名拼音含”zhang”的联系人	通过：张姓联系人均出现
声	输入”zw”	列出声母为 zw 的联系人	通过：张伟出现在结果中

测试项	输入数据	预期结果	实际结果 / 结论
母搜索		(如“张伟”)	
无结果搜索	输入“zzzzz”	列表为空, 显示“暂无联系人”	通过

4.6 导入导出测试

测试项	输入数据	预期结果	实际结果 / 结论
导出 CSV	菜单→导出 CSV, 选择路径保存	生成.csv 文件, Excel 可正常打开, 中文不乱码	通过: 文件字段完整, UTF-8 BOM 正确
导出 vCard	菜单→导出 vCard, 选择路径保存	生成.vcf 文件, 文件包含 BEGIN:VCARD... END:VCARD 标准格式	通过: 格式符合 vCard 3.0
导入 CSV	菜单→导入 CSV, 选择导出的 csv 文件	联系人追加到通讯录, 数量增加	通过: 重复导入不影响已有数据 (追加模式)
导入 vCard	菜单→导入 vCard, 选择.vcf 文件	解析所有联系人并追加, 姓名/手机/分组正确	通过: 与 iOS 导出的 vcf 兼容



测试结果截图

5 系统运行截图



主界面

新增联系人

基本信息 详细信息 所属分组

暂无头像
上传头像

姓名*

手机

电话

即时通信 QQ

电子邮箱

生日

工作单位

保存 取消

添加联系人界面

新增联系人

基本信息 详细信息 所属分组

暂无头像
上传头像

姓名*

手机

电话

即时通信 QQ

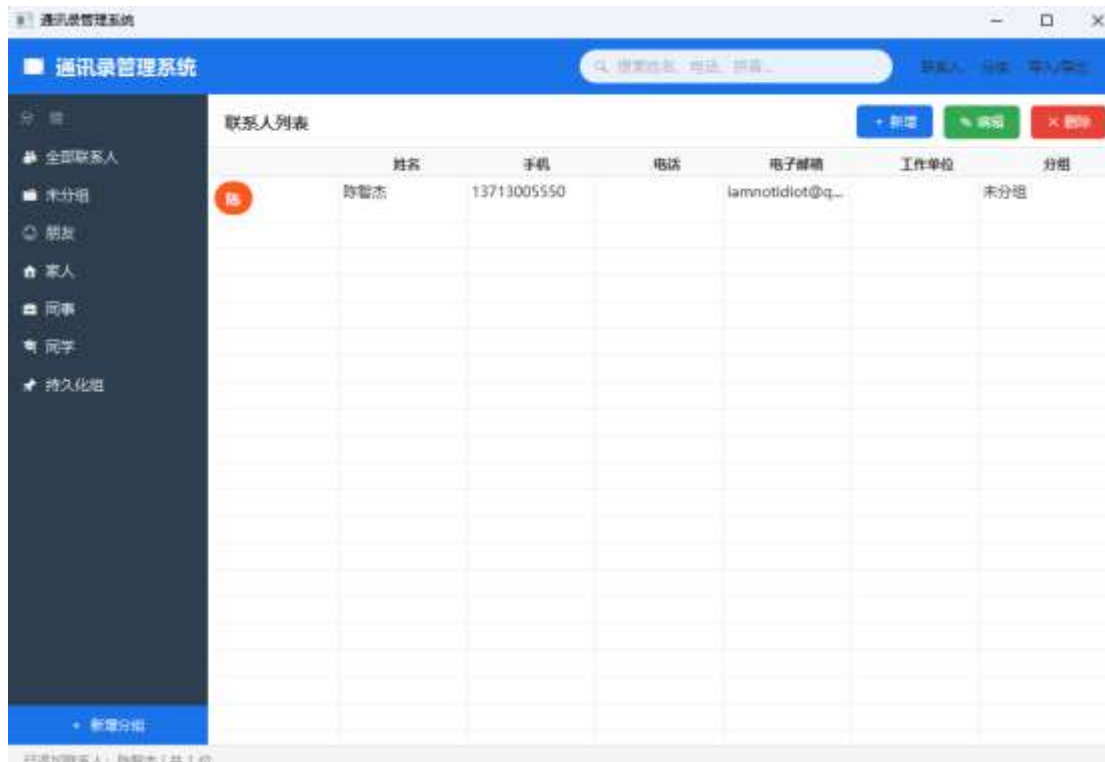
电子邮箱

生日

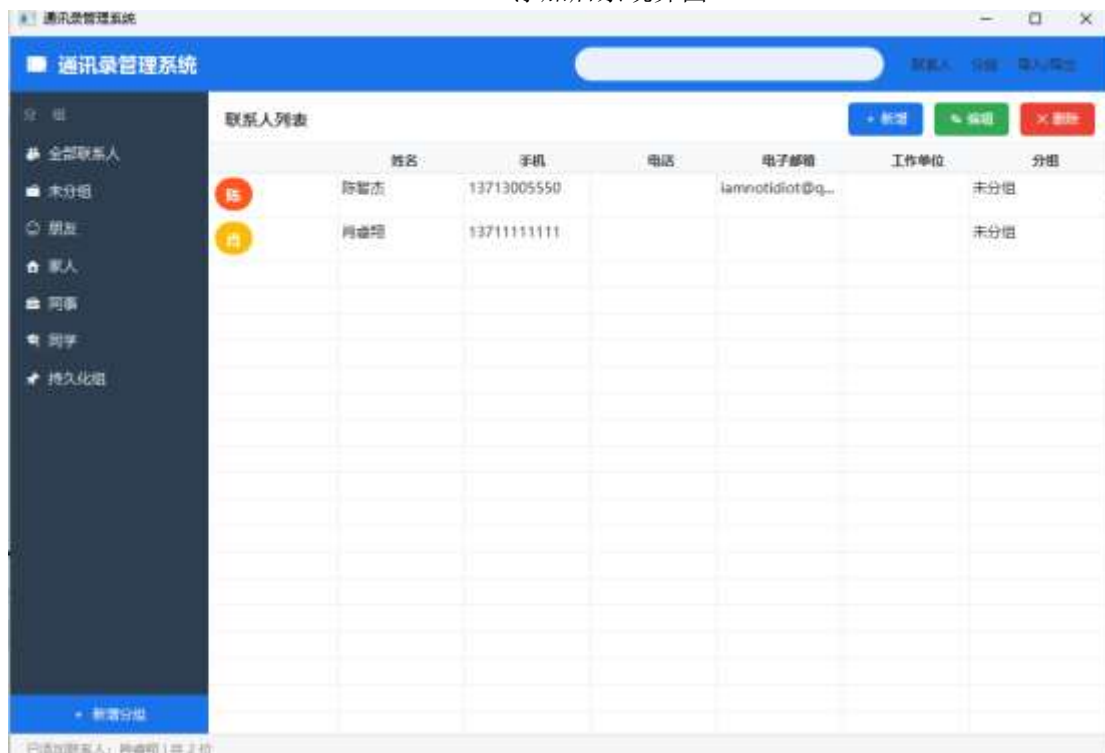
工作单位

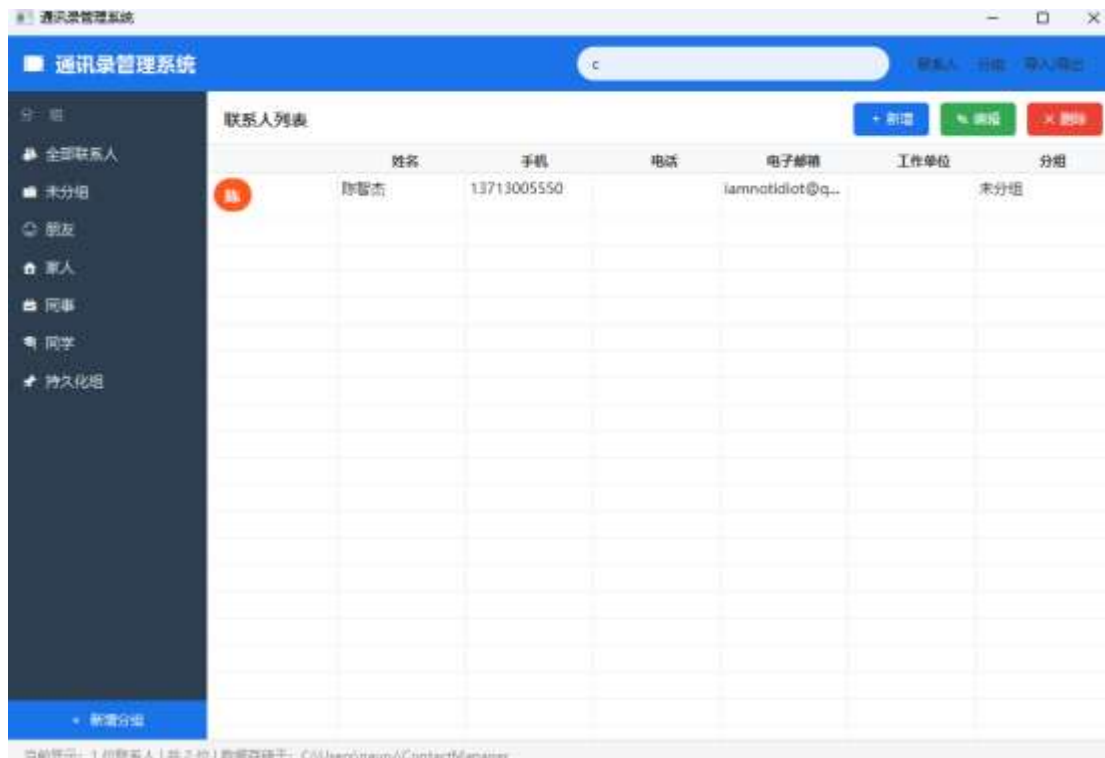
保存 取消

添加联系人



添加后系统界面





输入 c 后根据首字母模糊搜索到“陈智杰”

6 总结

6.1 个人承担工作

陈智杰 202425220506:

主导系统整体架构设计与业务模块开发，负责课程论文的主体撰写工作，并参与系统测试与稳定性验证。

肖睿翔 202425220524:

负责数据交互功能与界面开发实现，参与课程论文的修改完善，配合完成系统测试与优化调整。

6.2 课程设计体会

陈智杰 202425220506:

此次课设小组作业，巩固了 Java SE 知识，提高了项目架构设计知识，能画出各功能工作时的流程顺序图

肖睿翔 202425220524:

通过本次课设我从零熟悉 JavaFX 界面开发流程，掌握了可视化程序开发思路。通过全面测试优化程序细节，完善各项实用功能。此次合作开发让我积累了实战经验，也体会到团队合作的重要性，夯实了自身编程实践基础。